URBAN SENTINEL

UrbanIncident Fusion & Causal Intelligence

PRODUCT REQUIREMENTS DOCUMENT

A build-ready product blueprint for multi-source urban incident detection, evidence fusion, causal analysis, cascade-risk assessment, and response coordination.

Version 1.0 **Status** Baseline for implementation **Date** 12 October 2026

Primary context Urban operations / Mumbai-first demonstration, designed for extension to other cities

This PRD is intended to be the working contract between product, design, engineering, data, AI, and QA. It defines what must exist, how users move through it, how data is represented, and how acceptance will be judged.

Document note: this document consolidates the Urban Sentinel concept and implementation context available in this conversation. The attached research/architecture PDF could not be read by the document retrieval service in this run; source-specific statements not visible in the conversation are therefore not claimed as verified. Confirm any additional requirements in that source during product review.

1. Document control and how to use this PRD

Field	Definition
Product	Urban Sentinel / UrbanIncident Fusion & Causal Intelligence
Document owner	Product owner / project lead (to be assigned)
Audience	Product, UX/UI, backend, data engineering, AI/ML, DevOps, QA, demo/operator stakeholders
Status	Baseline v1.0; decisions marked TBD require explicit owner sign-off
Release target	MVP / hackathon-ready operational prototype; production hardening is a separate release gate
Normative language	MUST = required for MVP acceptance; SHOULD = expected unless a documented reason; MAY = optional/later

1.1 Product principle

Urban Sentinel combines weather, air-quality, traffic, water-level, and citizen-report signals into a shared operational picture. It helps an operator identify a likely incident, inspect supporting and contradictory evidence, understand plausible causes and downstream impacts, and prioritize response options. It is decision support—not an autonomous authority for emergency dispatch.

1.2 How teams should use this document

• Design uses Sections 5–6 to create navigation, page layouts, components, states, and responsive behavior.

• Backend/data uses Sections 7–10 for canonical event fields, ingestion contracts, deterministic processing, APIs, and persistence.

• AI/graph engineering uses Sections 8–9 for agent boundaries, evidence provenance, causal hypotheses, and cascade-risk outputs.

• QA uses Section 12 to create test cases and release gates; a feature is not complete until its acceptance criteria pass.

• Project leadership uses Sections 3, 13, and 14 for priorities, sequencing, risks, and unresolved decisions.

1.3 Explicit assumptions

ID	Assumption / constraint	Treatment in this PRD
A-01	Initial operating geography is Mumbai / India for demo scenarios.	Use configurable geographic bounds; do not hard-code geography into core services.
A-02	Some data providers may have access limits, quotas, or unavailable history.	Every connector supports live, historical, or synthetic/replay mode with visible provenance.
A-03	The first release is a human-supervised decision-support tool.	No automatic dispatch, public warning, or control-system actuation.
A-04	A source may be delayed, duplicated, contradictory, or offline.	The system must preserve raw events and expose source health and uncertainty.
A-05	Causal links can be hypotheses rather than proven facts.	UI and API must label hypothesis, evidence-supported association, and validated relationship separately.

2. Product summary, problem, and outcomes

2.1 Problem statement

Urban incident response is fragmented across data sources that differ in update frequency, location precision, semantics, reliability, and latency. Operators must manually determine whether signals refer to the same incident, whether a signal is trustworthy, what may have caused the event, what infrastructure or services could be affected next, and which intervention should be considered first.

2.2 Product vision

Provide a continuously updated, explainable incident picture that fuses heterogeneous signals into incident candidates and incident records, keeps competing explanations visible, estimates cascading risk, and gives operators an auditable basis for prioritization.

2.3 Goals and measurable outcomes

Goal	MVP success measure / proposed target	Measurement method
Unify source data	At least five source categories represented: weather, air quality, traffic, water, citizen reports.	Connector and replay test report.
Make event handling reliable	Duplicate replays do not create duplicate canonical events; late events are accepted and reconciled.	Automated idempotency and out-of-order tests.
Explain incident fusion	Every incident candidate shows why signals were matched and what evidence contradicts the match.	UI inspection + API assertions.
Expose uncertainty	Validity, incident membership, and cause confidence are distinct fields and displayed separately.	Schema and UX acceptance review.
Support triage	Operator can move from overview → map/list → incident detail → evidence → recommended response.	Usability walkthrough.
Make data quality visible	Connector freshness, last success, error state, and missing-signal context are visible.	Simulated source failure test.
Audit decisions	Material operator actions and system-generated changes have timestamps and actor/source attribution.	Audit log test.

2.4 Non-goals for MVP

• Fully autonomous emergency dispatch or direct control of traffic lights, pumps, gates, or public alert systems.

• Claiming causal proof solely because two events are correlated in time or space.

• Replacing official emergency, meteorological, water, or public-health authorities.

• Processing every raw sensor record through an LLM. Deterministic code handles validation, deduplication, anomaly checks, and candidate retrieval.

• Unlicensed collection or use of CCTV, private location data, or restricted partner feeds.

2.5 Primary value proposition

From fragmented signals to an explainable incident: what is happening, what supports the conclusion, what conflicts with it, what could happen next, and what action is worth considering.

3. Users, roles, and core journeys

Role	Primary needs	Allowed actions (MVP)
Operations analyst	Monitor live situation, triage incidents, compare evidence, prioritize.	View/filter, inspect, annotate, create/merge/split candidate with reason, acknowledge.
Incident commander / supervisor	Understand impact, confidence, escalation and response options.	Review incident summary, approve/decline suggested action, assign owner/status.
Data/platform operator	Diagnose stale feeds, connector failures, anomalous sensors, ingestion lag.	View source health, trigger authorized sync/replay, mark source maintenance, inspect errors.
System administrator	Manage users, source settings, thresholds, retention and access.	Configure roles and approved settings; changes audited.
Evaluator / demo viewer	Understand product behavior without privileged control.	Read-only dashboard, scenarios, evidence explanations, source provenance.

3.1 End-to-end operator journey

1. Open Overview. Review active incidents, severity, source freshness, and top cascade risks.

2. Select an incident marker or candidate row on Live Operations. Apply area, time, type, and status filters.

3. Open Incident Detail. Confirm what is known, confidence dimensions, timeline, affected assets, and unresolved questions.

4. Open Evidence & Causal Analysis. Inspect supporting/contradictory signals and compare competing hypotheses.

5. Open Cascade & What-if. Review possible downstream impacts and compare interventions with assumptions stated.

6. Create or review a response action. Record owner, priority, rationale, status, and approval where required.

7. Return to overview. Verify status changes and ensure the action is represented in the audit trail.

3.2 Role and permission baseline

Capability	Viewer	Analyst	Supervisor	Admin
View dashboard, map, incidents	Yes	Yes	Yes	Yes
Inspect evidence / hypotheses	Yes	Yes	Yes	Yes
Annotate incident	No	Yes	Yes	Yes
Merge / split / resolve incident	No	Yes (reason required)	Yes	Yes
Approve response action	No	No	Yes	Yes
Change thresholds / connectors / users	No	No	No	Yes
View audit history	Read-only subset	Yes	Yes	Yes

4. Scope, priorities, and release boundaries

Priority	Definition	Examples
P0 — Must	Blocks core demo or undermines trust/safety if absent.	Canonical event schema, ingestion provenance, deduplication, overview, map/list, incident detail, evidence panel, source health, basic cascade scenario, audit trail.
P1 — Should	Materially improves operator utility and completeness.	Competing cause hypotheses, what-if comparisons, saved filters, response queue, asset dependency graph, exports.
P2 — Later	Advanced capability after MVP evidence and validation.	Predictive ML trained on labelled history, advanced optimization, multi-city tenancy, mobile app, external alert integrations.

4.1 MVP functional requirements

ID	Requirement	Priority	Acceptance intent
FR-001	Multi-source ingestion	MUST	Ingest weather, air-quality, traffic, water-level, and citizen-report events through adapters or replay files.
FR-002	Canonical event contract	MUST	Normalize source events to one versioned schema while retaining source payload/provenance.
FR-003	Validation	MUST	Reject or quarantine invalid timestamps, coordinates, units, required fields, and malformed payloads with reason.
FR-004	Idempotent persistence	MUST	Repeated delivery of the same source event must not create a second canonical event.
FR-005	Out-of-order support	MUST	Late events are retained and incident associations can be reconciled without losing history.
FR-006	Anomaly detection	MUST	Produce explainable anomaly records from configured thresholds/baselines and sensor health rules.
FR-007	Candidate matching	MUST	Retrieve candidate incidents by time, distance, type compatibility, and asset/road/drain relationships.
FR-008	Evidence-aware fusion	MUST	Show supporting and contradicting evidence, match score components, and unresolved alternatives.
FR-009	Operator incident actions	MUST	Support acknowledge, annotate, status transition, and merge/split with reason and actor.
FR-010	Source health	MUST	Show freshness, last success, error state, ingestion count/lag, and provenance mode.
FR-011	Cascade risk	MUST	Represent at least one configurable dependency chain and expose assumptions/impact outputs.
FR-012	Response recommendations	MUST	Rank response options with rationale, estimated impact, uncertainty, and human approval status.
FR-013	Auditability	MUST	Record material system/operator changes with time, actor, object, action, and reason.
FR-014	Security	MUST	Protect secrets, enforce role permissions for write actions, and avoid logging credentials/PII.
FR-015	Demo/replay mode	MUST	Run a deterministic scenario when live provider access is unavailable; clearly label synthetic/historical data.
FR-016	Search/filter/export	SHOULD	Filter incidents/events by time, geography, type, status, source, severity; export a bounded result set.

5. Information architecture and page count

Recommended MVP: **9 primary application pages** plus authentication/utility states. This is enough to demonstrate the complete operator workflow without spreading core functionality across too many screens.

#	Page	Primary question answered	Main users
1	Overview Dashboard	What needs attention right now?	All roles
2	Live Operations Map	Where are incidents and related signals?	Analyst, supervisor
3	Incident Detail	What do we know about this incident?	Analyst, supervisor
4	Evidence & Causal Analysis	Why do we think signals belong together; what else could explain them?	Analyst, supervisor
5	Cascade Risk & What-if	What could happen next and which intervention may help?	Supervisor, analyst
6	Response & Action Center	What actions are pending, assigned, or completed?	Analyst, supervisor
7	Event Explorer	What raw/normalized signals were received and how were they processed?	Analyst, platform operator
8	Data Sources & Health	Are feeds fresh, reliable, and correctly configured?	Platform operator, admin
9	Settings & Audit	Who can change what, which thresholds apply, and what changed?	Admin, supervisor

5.1 Global shell

• Left navigation rail: Overview, Live Operations, Incidents, Evidence & Causal Analysis, Cascade Risk, Response Center, Event Explorer, Data Sources, Settings & Audit.

• Top bar: city/operating area selector, time window, global search, data mode badge (LIVE / HISTORICAL / SYNTHETIC), notification indicator, user/role menu.

• Persistent system banner: show degraded data, stale feeds, replay mode, or incident-processing backlog. Banner must never be hidden behind a chart.

• Main content area: page title, concise purpose, primary action, filters, content region, and a consistent right-side detail drawer where it reduces navigation.

• Use accessible severity labels and icons as well as color. Do not communicate criticality through color alone.

5.2 Routes

Route	Page / behavior
/login	Authentication; role-aware redirect; demo access only if explicitly enabled.
/overview	Operational summary dashboard.
/operations	Map + synchronized incident/event list.
/incidents/:incidentId	Incident record and status/actions.
/incidents/:incidentId/evidence	Evidence graph and causal hypotheses.
/incidents/:incidentId/cascade	Cascade graph, scenario assumptions, intervention comparison.
/response	Response action queue and action detail.
/events	Event explorer, processing trace, raw payload access by permission.
/sources	Source catalog, health, provenance mode, connector status.
/settings	Users/roles, thresholds, audit history, system configuration.

5.3 Page-count decision

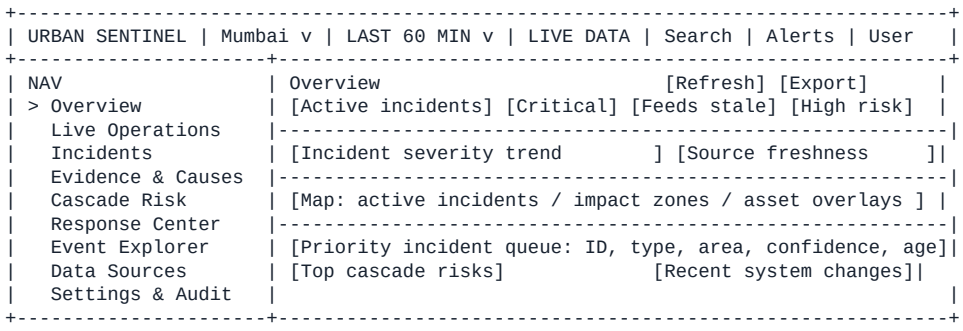
Count nine primary pages for planning. Login, not-found, access-denied, empty, loading, error, and confirmation states are utility states, not additional primary pages. Incident Detail, Evidence, and Cascade are separate routes because they support different tasks and can be deep-linked.

6. Screen specifications and low-fidelity wireframes

6.1 Page 1 — Overview Dashboard

Purpose: Let an operator assess operational risk in under 30 seconds and identify the next best place to investigate.

Layout sketch — Overview



Conceptual low-fidelity wireframe; final visual design may evolve while preserving the specified information hierarchy.

Components and behavior

• KPI cards: active incidents, critical/high severity count, stale/degraded sources, high cascade-risk count. Each card opens a pre-filtered view.

• Trend chart: active incidents over selectable time window; hover reveals counts and source mode.

• Map: clustered markers, severity, freshness/uncertainty indicator; selecting a marker opens a preview and deep link.

• Priority queue: sortable by severity, risk, confidence, age, and last update; rows show separate event-validity, membership, and cause confidence when available.

• Source freshness panel: last successful fetch, latest event time, error/lag status, and live/replay mode.

• Top cascade risks: concise chain, affected assets/area, risk band, confidence, and time horizon.

Actions

• Open incident; change time range; filter operating area; refresh; export current dashboard summary; acknowledge a notification.

Acceptance criteria

• Dashboard shows loading, empty, partial-data, and error states distinctly.

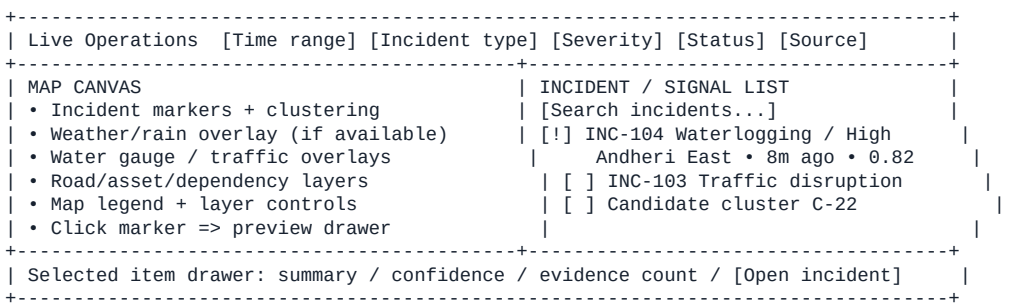
• Every KPI can be traced to a defined query and time window.

• Data mode is visible on the page and remains visible on exported snapshots.

• A source outage does not blank unrelated panels; affected panels show degraded-data messaging.

6.2 Page 2 — Live Operations Map

Layout sketch — Operations Map



Conceptual low-fidelity wireframe; final visual design may evolve while preserving the specified information hierarchy.

Requirements

• Map and list filters are synchronized. Selecting a list row highlights the map object; selecting a marker selects the row.

• Layers are individually toggled and clearly label unavailable data. Do not display a blank overlay as if it were a zero reading.

• Candidate clusters are visually distinct from confirmed/operator-reviewed incidents.

• Support bounded map navigation, marker clustering, time slider/playback for replay scenarios, and a clear reset-filters control.

• If geolocation is approximate or missing, show an uncertainty/area representation instead of a falsely precise pin.

Acceptance criteria

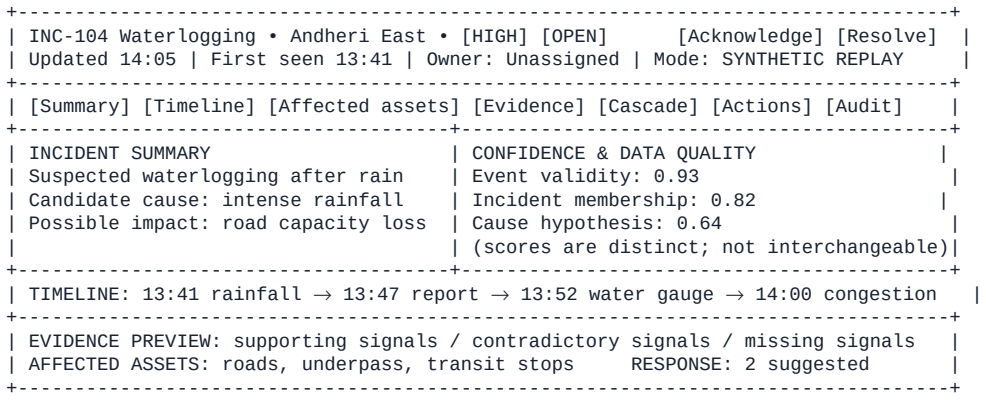
• A user can locate and open any listed incident in two interactions or fewer.

• The map displays data timestamp and mode.

• Map failure falls back to the synchronized list.

6.3 Page 3 — Incident Detail

Layout sketch — Incident Detail



Conceptual low-fidelity wireframe; final visual design may evolve while preserving the specified information hierarchy.

Fields that must be present

Field group	Required fields
Identity	Incident ID, title/type, canonical status, severity, created/first-seen/last-updated timestamps.
Location	Area/geometry, coordinate precision or uncertainty, impacted radius/segment, affected asset references.
Confidence	Event validity, incident membership, cause-hypothesis confidence, plus explanation/calibration status.
Evidence summary	Supporting count, contradictory count, missing expected signals, source diversity, last evidence time.
Causal summary	Leading hypothesis, alternatives, assumptions, causal links and evidence references.
Impact	Affected road/asset/transit/service, estimated extent, time horizon, impact confidence.
Operations	Owner, status, severity, notes, tags, action queue, audit history.

State transitions

Candidate → Open → Acknowledged → Investigating → Mitigating → Resolved → Closed. Reopen is allowed with a required reason. Merge/split are explicit audited operations, never silent destructive edits. Teams may simplify statuses for MVP, but the allowed transition graph must be defined in code.

Acceptance criteria

• Changing status requires permission and creates an audit entry.

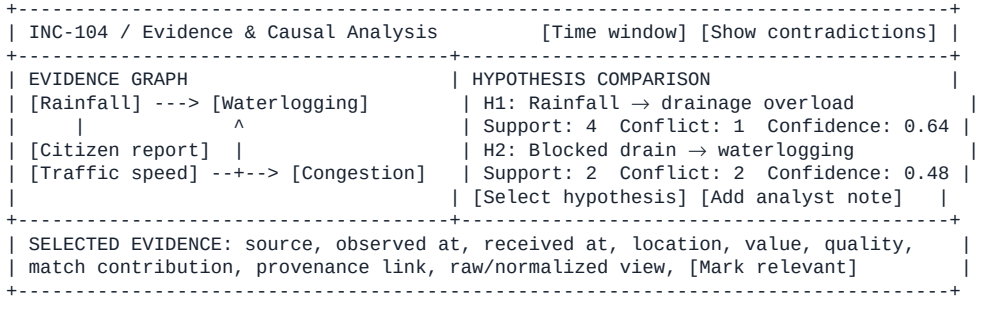
• Incident detail links to evidence, cascade, and response pages while preserving incident ID.

• Resolution records time, actor, and optional outcome/notes.

• The screen never presents a heuristic score as a calibrated probability unless calibration has been evaluated and documented.

6.4 Page 4 — Evidence & Causal Analysis

Layout sketch — Evidence & Causal Analysis



Conceptual low-fidelity wireframe; final visual design may evolve while preserving the specified information hierarchy.

Requirements

• Evidence nodes represent events, reports, sensor readings, assets, hypotheses, and incidents; edges carry relationship type, provenance, timestamp, and confidence/strength where applicable.

• Show both supporting and contradicting evidence. Provide a filter to isolate either set and a legend for inferred versus observed relationships.

• Each hypothesis includes claim, supporting evidence, counter-evidence, assumptions, confidence, generated/updated time, and status (candidate/reviewed/rejected/validated).

• Selecting an evidence node opens source metadata, event time vs ingest time, source mode, validation/anomaly flags, and the normalized payload. Raw payload access is permission-controlled.

• An analyst can mark evidence relevant/irrelevant and add a rationale; this changes annotation state, not the immutable source event.

• Do not render an edge as proven causality merely because it is present in the graph. Label relationship semantics explicitly.

Acceptance criteria

• A hypothesis cannot be shown without evidence references or an explicit 'insufficient evidence' reason.

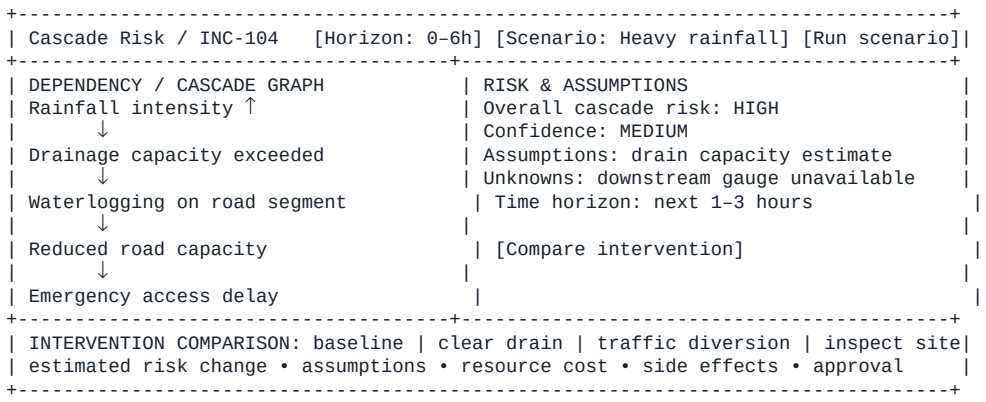
• Contradictory evidence remains discoverable.

• Graph is usable with keyboard-accessible equivalent list/table view.

• The page separates correlation, inferred causal hypothesis, and validated causal relationship.

6.5 Page 5 — Cascade Risk & What-if

Layout sketch — Cascade Risk



Conceptual low-fidelity wireframe; final visual design may evolve while preserving the specified information hierarchy.

Requirements

• Represent dependencies among hazards, infrastructure, transport capacity, and service impacts as an explicit graph.

• Allow a user to select a scenario/horizon and compare a baseline with at least two interventions in MVP.

• Each result lists inputs, assumptions, unknowns, estimated risk band/change, and a human-readable explanation.

• Interventions are recommendations for review; they are not sent to real-world control systems.

• Graph data must distinguish configured/known dependencies from scenario assumptions and relationships supported by observed data.

MVP scenario example

Rainfall intensifies → drainage capacity is exceeded → waterlogging affects a road segment → effective road capacity falls → congestion increases → emergency access time may rise. Each arrow must have a type and provenance/assumption label. The product must not claim this chain is universally true for all locations.

Acceptance criteria

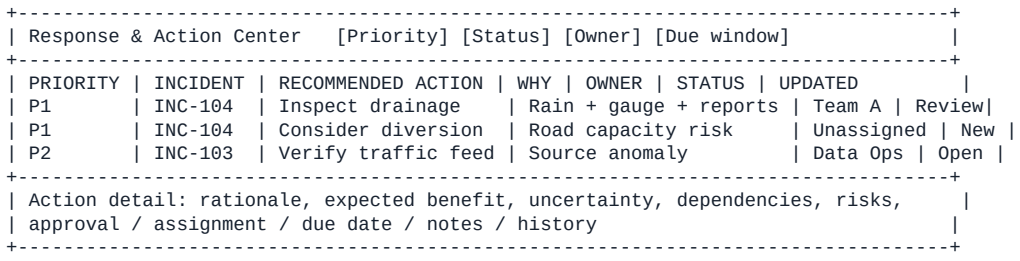
• Scenario results are reproducible for the same inputs/version.

• Missing dependency data lowers certainty or is surfaced as unknown; it is not silently imputed as safe.

• Each comparison identifies the baseline and the changed intervention assumptions.

6.6 Page 6 — Response & Action Center

Layout sketch — Response Center



Conceptual low-fidelity wireframe; final visual design may evolve while preserving the specified information hierarchy.

Requirements

• Create a proposed action from an incident or recommendation; store owner, priority, due time, status, rationale, expected impact, uncertainty, and originating evidence.

• Statuses: Proposed, Needs Review, Approved, Assigned, In Progress, Completed, Rejected, Cancelled.

• Supervisor approval is required for actions configured as high impact. The MVP must not auto-dispatch or externally execute actions.

• Show duplicate/similar actions and affected incident to reduce conflicting response work.

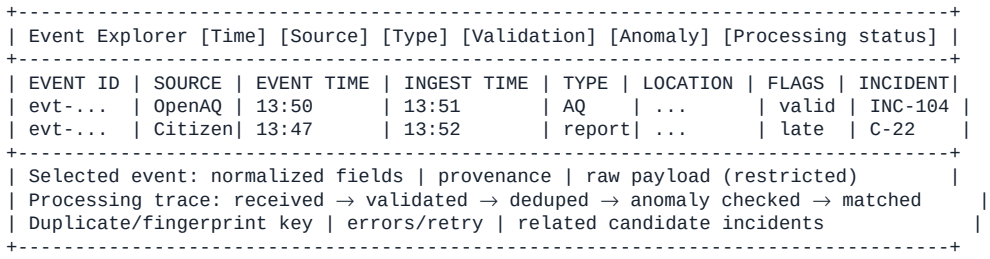
- Every approval, rejection, reassignment, or completion is audited.

Acceptance criteria

- An action always links to its incident and rationale.
- Approval permissions are enforced server-side.
- Action state persists across refresh and appears in the audit history.

6.7 Page 7 — Event Explorer

Layout sketch — Event Explorer



Conceptual low-fidelity wireframe; final visual design may evolve while preserving the specified information hierarchy.

Requirements

• Search by event ID, source event ID, source, type, time range, location, anomaly flag, validation outcome, and incident association.

• Display event time and ingest time separately to make latency and out-of-order arrival visible.

• Processing trace is append-only for material steps and includes result, duration, reason, and processor version where feasible.

• A duplicate delivery is linked to the canonical event/receipt record and is not counted as independent corroboration.

• Raw payload is shown only to authorized users; secrets and unnecessary personal information are redacted.

Acceptance criteria

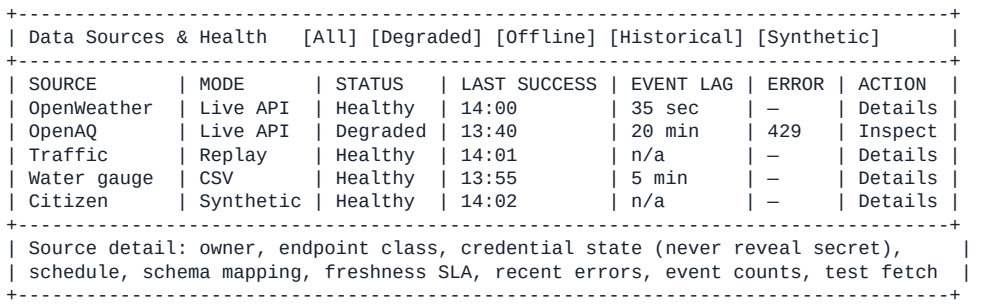
• Filters are composable and resettable.

• An invalid/quarantined event has a visible reason and remediation status.

• A user can navigate from event to associated incident and back.

6.8 Page 8 — Data Sources & Health

Layout sketch — Data Sources



Conceptual low-fidelity wireframe; final visual design may evolve while preserving the specified information hierarchy.

Requirements

• Catalog each source with owner, data class, connector type, operating mode, configured cadence, freshness expectation, last success/failure, and error category.

• Never display API keys or full secrets. Show configured/missing/invalid state only.

• Health states: Healthy, Degraded, Stale, Offline, Misconfigured, Disabled, Replay/Synthetic (mode is separate from health).

• Source failures are isolated: one failed adapter must not stop unrelated adapters or API reads.

• Expose missing expected signals as an observability/data-quality concern, not as evidence that the city is safe.

Acceptance criteria

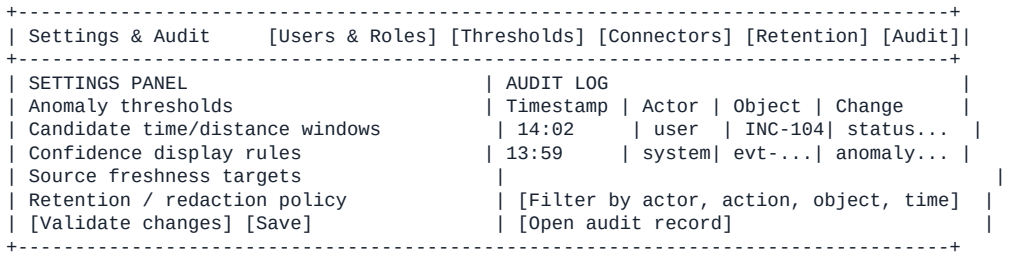
• An offline feed can be simulated and visibly changes status.

• A replay/synthetic source is unmistakably labeled in all relevant views.

• Operator can see recent failures and a safe remediation instruction.

6.9 Page 9 — Settings & Audit

Layout sketch — Settings & Audit



Conceptual low-fidelity wireframe; final visual design may evolve while preserving the specified information hierarchy.

Requirements

• Role administration supports least privilege; changes to roles are audited.

• Thresholds are versioned, validated, and take effect with a recorded effective time. Show current value and unit.

• Connector settings are managed by secret references/environment configuration, not stored in plaintext in ordinary settings records.

• Audit records include actor/system identity, timestamp, object type/ID, action, before/after summary where appropriate, reason, and request/correlation ID.

• Retention and export controls must comply with configured privacy/legal requirements. Exact retention durations are TBD pending owner approval.

Acceptance criteria

• Unauthorized users cannot save settings even if they call the API directly.

• Invalid threshold combinations are rejected with actionable errors.

• Audit entries cannot be edited through normal application workflows.

7. Data requirements and canonical event contract

7.1 Supported source categories

Source category	Example data	MVP ingestion mode	Important handling
Weather	Rainfall/precipitation, temperature, humidity, weather condition, observation time, coordinates.	Live API where credentials/access allow; otherwise replay.	Preserve accumulation window/unit; do not treat period rainfall as instantaneous intensity.
Air quality	Pollutant measurements, AQI where supplied, station ID, station location, observation time.	Live API / historical export / replay.	Retain pollutant, unit, averaging period, station metadata, quality flags.
Traffic	Speed, congestion, travel-time ratio, road segment, observation time.	Authorized API / sample feed / replay.	Normalize road segment references; current speed/free-flow ratio only if denominator is available.
Water	Water level, gauge/station, threshold/flood stage, observation time.	Official dataset/API where available; CSV/replay.	Record datum/unit, station, reporting interval, and threshold provenance.
Citizen reports	Incident type, text/category, time, location/area, optional attachment reference.	Manual form / synthetic scenario / authorized feed.	Treat as unverified report; minimize PII; separate reported time from submission time.
Infrastructure / map	Road segments, drainage assets, transit stops, dependency links, geometry.	OSM/GTFS/approved city datasets/manual seed.	Store source/licence/last updated; map availability does not prove operational status.

7.2 Canonical event schema — required fields

Field	Type / example	Rule
event_id	UUID/string	Stable internal ID; immutable.
source_id	string	Configured source identity.
source_event_id	string|null	Provider ID when available.
event_type	enum/string	Canonical taxonomy plus source subtype.
event_time	UTC timestamp	When observation/report occurred; required or explicitly unknown.
ingest_time	UTC timestamp	When platform received it; server-generated.
location	lat/lon, geometry, accuracy	Coordinates validated; location may be null with reason/area reference.
measurements	array of name/value/unit/period	Typed values; units normalized and original unit preserved.
source_kind	live_api / historical / synthetic / manual	Required provenance shown to users.
quality	status + flags	Valid, suspect, invalid, stale, imputed (imputation must be explicit).
payload_ref / raw_payload	JSON or object reference	Retain per policy; redact secrets/PII; raw and normalized data separable.
schema_version	string	Supports safe evolution and migration.
idempotency_key	string/hash	Supports atomic duplicate protection.

7.3 Related entities

• **Incident:** incident_id, type, status, severity, geometry/area, created_at, updated_at, owner, confidence dimensions, summary, resolution.

• **EvidenceLink:** link_id, incident_id, event_id, relation (supports/contradicts/context), score contribution, rationale, created_at, processor/version.

• **Hypothesis:** hypothesis_id, incident_id, claim, status, confidence, assumptions, evidence IDs, counter-evidence IDs, version, updated_at.

• **Anomaly:** anomaly_id, event/source/asset reference, rule/baseline, observed value, expected range, severity, explanation, detection time.

• **SourceHealth:** source_id, mode, status, last attempt/success, event time watermark, ingest lag, error class, count, freshness SLA.

• **Action:** action_id, incident_id, type, rationale, owner, priority, due_at, status, approval, expected impact, audit references.

• **Asset / dependency:** asset_id, type, geometry, source, freshness, dependency edges, relationship semantics, provenance.

7.4 Data quality and provenance rules

• Always preserve observed_at/event_time and received_at/ingest_time separately.

• Retain source identity, source event ID, connector version, source mode, and original units/payload reference.

• A missing reading is not zero. A stale feed is not healthy. A synthetic event is not live evidence.

• Duplicate delivery must not count as independent corroboration; keep delivery receipt/duplicate trace if operationally useful.

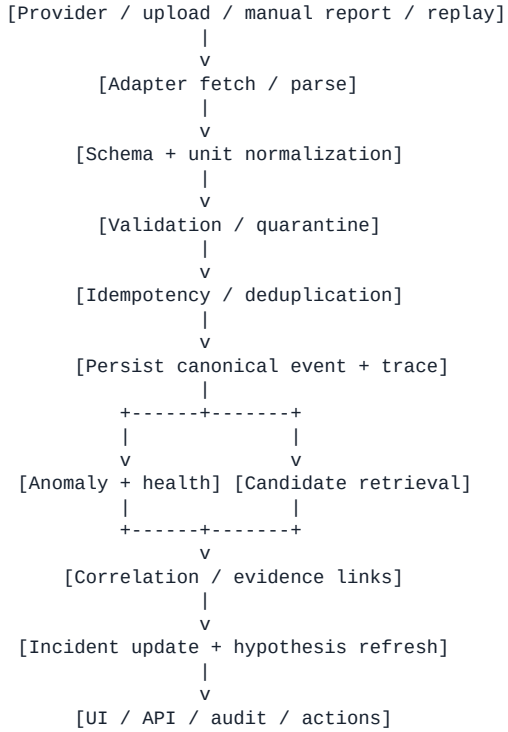
• Contradictory signals remain visible and affect explanation/uncertainty rather than being silently discarded.

• Use UTC internally; render local time with timezone label in the UI. Preserve source-reported timezone where available.

8. Ingestion and deterministic processing requirements

8.1 Processing sequence

Required processing flow



Conceptual low-fidelity wireframe; final visual design may evolve while preserving the specified information hierarchy.

8.2 Validation

• Validate required fields, timestamp parseability, coordinate bounds, finite numeric values, allowed event types, measurement units, and payload size.

• Classify outcomes as accepted, accepted-with-warning, quarantined, or rejected. Store a machine-readable reason code and a human-readable explanation.

• Do not silently discard malformed records. Quarantine supports inspection and safe replay after mapping correction.

• Normalize units only when conversion is defined; preserve original value/unit and conversion metadata.

8.3 Deduplication and idempotency

Use a hierarchy: (1) provider/source event ID, (2) explicit idempotency key, (3) fallback content fingerprint. Enforce uniqueness atomically in the persistence layer. A read-before-write check alone is not sufficient under concurrent ingestion.

• Fingerprint inputs should include normalized source ID, event type, event time bucket/precision, location precision, and stable measurement fields.

• Fingerprint deduplication is a fallback, not universal truth: coordinate/time rounding may merge distinct events or miss near-identical duplicates. Make rules source-specific and configurable.

• Record duplicate receipt, canonical event reference, matching rule, and decision time.

• Do not use a duplicate as additional evidence when calculating source diversity or corroboration.

8.4 Anomaly detection

Rule family	Example	Output required
Domain threshold	Water level crosses configured alert threshold.	Observed value, threshold, unit, rule ID, severity, provenance.
Robust baseline	Traffic speed unusually low vs recent history.	Baseline window, median/MAD or method, score, sample sufficiency.
Rate-of-change	Water level rises faster than configured rate.	Previous/current values, elapsed time, calculated rate, threshold.
Sensor health	Flatline, impossible jump, stale heartbeat, repeated identical payload.	Health state, last valid reading, reason, recommended check.
Cross-signal inconsistency	Nearby sensors disagree beyond tolerance.	Signals compared, distance/time windows, contradiction explanation.

Robust z-score may be implemented as (x – median) / (1.4826 × MAD), with explicit handling for MAD=0, insufficient history, and nonstationary data. Anomaly scores are not incident severity by themselves.

8.5 Candidate retrieval and matching

Retrieve candidates using indexed time windows, spatial proximity/geofencing, compatible event types, shared asset/road segment, and known infrastructure dependencies. Apply a transparent weighted score, for example:

S(event, incident) = w_time·T + w_space·G + w_type·K + w_asset·A + w_quality·Q

• T = time compatibility; G = spatial compatibility; K = event/incident type compatibility; A = shared asset/dependency; Q = source/event quality adjustment.

• Return score components and rule/version, not only a single opaque number.

• Scores are heuristic rankings until calibrated against labelled incidents; do not label them probabilities without validation.

• Keep multiple candidates when ambiguity is meaningful. Support candidate, linked, rejected, and needs-review outcomes.

• Late-arriving events trigger bounded reconciliation for relevant incidents; do not assume ingestion order equals event order.

8.6 Incident fusion policy

• Separate event validity, incident membership, and cause-hypothesis confidence.

• Fuse across independent source categories when evidence is compatible; do not equate many repeated readings from one source with diverse corroboration.

• Contradictory evidence can lower confidence, split a candidate, or create an alternative hypothesis; it must remain inspectable.

• Human override is allowed with reason and audit trail; the system retains the previous automated recommendation and provenance.

• Reproducible replay should produce stable event IDs/association outcomes for a fixed dataset, configuration, and software version.

9. AI agents, graph model, and causal intelligence

9.1 Agent architecture

Use a controlled, stateful orchestration approach (LangGraph is the recommended baseline) for tasks that benefit from contextual reasoning. Deterministic Python services remain authoritative for validation, deduplication, thresholds, persistence, and candidate retrieval. Do not call an LLM for every sensor event.

Component	Responsibility	Inputs	Outputs / guardrails
Orchestrator	Route tasks, manage state, retries, checkpoints, approval gates.	Incident state, processing result, requested analysis.	Versioned workflow state; no direct dispatch; explicit failure path.
Signal Intelligence specialist	Summarize anomalies, data quality, and signal reliability.	Validated events, anomaly results, source health.	Structured explanation with evidence IDs; cannot change raw events.
Incident Fusion specialist	Explain candidate membership and alternative groupings.	Candidate scores, event metadata, evidence links.	Recommended match/split/hold plus reasons; deterministic rules remain authoritative.
Causal/Cascade specialist	Generate competing causal hypotheses and trace potential cascade paths.	Incident graph, asset dependencies, observed/contradictory evidence.	Structured hypotheses, assumptions, unknowns, horizon; no unsupported proof claim.
Impact/Response specialist	Summarize affected services and rank response options.	Cascade outputs, impact data, action catalogue, constraints.	Ranked suggestions, rationale, trade-offs, approval requirement.

9.2 Structured output contract

• All agent outputs must be machine-readable, schema-validated, and include referenced event/asset/incident IDs.

• Include claim, evidence_for[], evidence_against[], assumptions[], unknowns[], confidence, confidence rationale, model/prompt version, generated_at, and status.

• If evidence is insufficient, return an explicit insufficient-evidence result rather than fabricating missing measurements or causal links.

• Use bounded retries, timeouts, and fallback summaries. Agent failure must not block core event ingestion.

• Treat provider payloads and citizen text as untrusted data; they must never be interpreted as system instructions.

9.3 Neo4j graph model

Node / edge	Key properties	Purpose
(:Event)	event_id, type, event_time, source_id, quality, source_kind	Graph representation of canonical event.
(:Incident)	incident_id, type, status, severity	Incident grouping and navigation.
(:Asset)	asset_id, type, geometry/ref, source, freshness	Road, drainage, gauge, transit or other infrastructure.
(:Hypothesis)	hypothesis_id, claim, status, confidence, version	Competing causal explanations.
(:Action)	action_id, type, status, priority	Suggested/approved response action.
(:Event)-[:SUPPORTS|CONTRADICTS|CONTEXT_FOR]->(:Incident)	relation, score, rationale, created_at, processor_version	Evidence membership with explanation.
(:Hypothesis)-[:SUPPORTED_BY|CHALLENGED_BY]->(:Event)	strength, provenance, rationale	Evidence behind and against a hypothesis.
(:Asset)-[:DEPENDS_ON|AFFECTS|CONNECTED_TO]->(:Asset)	relationship_type, source, assumption/validation status	Infrastructure dependency and cascade paths.
(:Incident)-[:AFFECTS]->(:Asset)	impact_type, confidence, time_horizon	Incident footprint and consequences.

9.4 PostgreSQL/PostGIS vs Neo4j

Store	System of record responsibility
PostgreSQL/PostGIS	Canonical events, incidents, action status, source health snapshots, user/role references, audit events, spatial/time queries, idempotency constraints, transactional updates.
Neo4j	Relationship traversal, evidence graph, hypothesis links, infrastructure dependencies, cascade-path exploration.
Object/file storage (optional)	Large raw payloads or attachments under retention and access policies; database stores a controlled reference.

PostgreSQL is the operational source of truth. Graph writes should be idempotent and reconcilable. If updates span both stores, use an outbox/retry/reconciliation pattern rather than assuming a distributed transaction.

9.5 Causal semantics

• Observed temporal/spatial co-occurrence is not proof of causality.

• Every relationship must be labeled as configured dependency, observed association, inferred hypothesis, or validated relationship.

• A cascade path must expose its assumptions, unknown inputs, time horizon, and supporting data.

• Causal confidence is distinct from incident-membership confidence and should be calibrated/evaluated separately.

10. API, persistence, and integration requirements

10.1 MVP API surface

Method + route	Purpose	Required behavior
GET /health	Liveness/readiness summary	No secrets; report service state and optional dependency status.
POST /api/v1/events	Accept normalized/manual events	Validate, enforce idempotency, return canonical event or duplicate reference.
POST /api/v1/ingest/{source}	Trigger permitted source ingestion	Auth/role check, per-source rate limit, safe error handling, run ID.
GET /api/v1/events	Query event explorer	Bounded pagination; filters; safe fields by role.
GET /api/v1/events/{id}	Event detail + processing trace	Return normalized data, provenance, quality, associated incidents.
GET /api/v1/sources/health	Source status dashboard	Freshness, mode, last success, lag, error category.
GET /api/v1/anomalies	Anomaly list	Filter by time, source, type, severity, status.
GET /api/v1/incidents	Incident list/map data	Filters, pagination, separate confidence dimensions.
GET /api/v1/incidents/{id}	Incident detail	Summary, status, evidence summary, affected assets, actions.
GET /api/v1/incidents/{id}/evidence	Evidence graph data	Supporting and contradictory edges, provenance, scores.
GET /api/v1/incidents/{id}/cascade	Cascade graph and scenario	Assumptions, horizon, outputs, model/config version.
POST /api/v1/incidents/{id}/actions	Create action	Validate permissions and required rationale/owner/priority.
PATCH /api/v1/actions/{id}	Update action state	Allowed state transitions and audit record.
GET /api/v1/audit	Audit search	Role-protected, paginated, filters by actor/object/time/action.

10.2 API standards

• Version API paths and schema payloads; document them in OpenAPI.

• Use consistent error envelopes: code, message, request_id, field_errors (where relevant), retryable flag.

• Use UTC ISO-8601 timestamps, stable IDs, bounded pagination, and explicit sorting.

• Return 4xx for invalid/unauthorized requests; 5xx for unexpected server faults. Never expose stack traces or secrets to clients.

• Protect ingestion and mutating endpoints with authentication, role checks, rate limits, and audit logging.

• Make long-running ingest/scenario operations return a run ID and status rather than blocking indefinitely.

10.3 Suggested service structure

```
app/
  main.py
  core/          config.py, logging.py, security.py
  schemas/       event.py, incident.py, action.py, source.py
  api/           events.py, sources.py, anomalies.py, incidents.py, actions.py, audit.py
  ingestion/     base.py, openweather.py, openaq.py, traffic.py, water.py, citizen.py, replay.py
  processing/    normalize.py, validate.py, deduplicate.py, anomaly.py, candidates.py, correlation.py
  services/      event_service.py, source_health.py, incident_service.py, cascade_service.py
  db/            models.py, session.py, repositories.py, migrations/
  graph/         neo4j_client.py, graph_writer.py, evidence_queries.py
  agents/        graph.py, schemas.py, signal_agent.py, fusion_agent.py, causal_agent.py, response_agent.py
  tests/         unit/, integration/, replay/, api/
```

10.4 Persistence rules

• Use database uniqueness constraints for source event IDs/idempotency keys where possible.

• Keep immutable event facts separate from mutable incident/action state.

• Use migrations and seed fixtures; do not rely on manual database edits for normal setup.

- Store timestamps for creation/update and actor/source provenance on mutable records.
- Provide backup/restore and migration verification before any production pilot.

11. UX, visual design, and interaction standards

11.1 Visual direction

Recommended visual language: high-clarity operations console, restrained navy/blue base, neutral surfaces, teal for normal/healthy, amber for caution, red for critical. Keep density appropriate for desktop operations. The map and incident queue are primary working surfaces; decorative animation should never compete with alert comprehension.

Element	Requirement
Typography	One UI sans-serif family; clear hierarchy; dense data tables remain readable at normal desktop zoom.
Color	Use severity colors consistently, paired with text labels/icons; support color-blind users.
Severity	Define an approved scale (e.g., Critical/High/Moderate/Low/Info) and document rules. Threshold values are configuration, not hard-coded visual assumptions.
Confidence	Display named dimension + numeric/qualitative value + explanation; avoid a single blended trust badge.
Time	Show local time with timezone where needed; distinguish observation time from ingestion time.
Map	Legend, layer toggles, selection state, clustering, uncertainty geometry, and accessible list alternative.
Charts	Show units, aggregation window, source freshness, and no-data states. Avoid implying missing values are zero.
Notifications	Prioritize actionable changes; avoid repeated notifications for the same deduplicated signal.
Responsive	Desktop-first (1440×900 baseline); usable tablet layout; mobile read-only fallback is acceptable for MVP.

11.2 Global UI states

State	Required UX
Loading	Skeleton/loader and contextual label; do not flash misleading zero values.
Empty	Explain why there is no data and how to broaden filters or load a replay scenario.
Partial/degraded	Show which sources/panels are affected and the last successful timestamp.
Error	Human-readable message, request ID, retry where safe, preserve user filters.
Permission denied	Explain missing permission without exposing restricted data.
Stale	Show staleness threshold and last observation; do not style as healthy.
Synthetic/replay	Persistent visible mode badge, scenario name, and time control.
Confirmation	Require confirmation for destructive or high-impact changes; capture reason for merge/split/resolve where applicable.

11.3 Accessibility

• Target WCAG 2.2 AA as the design target; verify keyboard navigation, focus visibility, contrast, semantic headings, and screen-reader labels.

• Provide a tabular/list alternative to graph-only evidence and map-only incident access.

• Never use color as the only differentiator; include icons, labels, patterns, or text.

• All critical actions must be reachable without drag-only interaction.

12. Non-functional requirements, security, and privacy

Area	MVP requirement / target	Validation
Availability / resilience	Core read APIs and dashboard remain available if one external source fails; retries are bounded.	Simulate provider outage and verify unrelated flows.
Performance	Proposed target: common dashboard/API reads p95 < 2 seconds on demo dataset; candidate retrieval p95 < 3 seconds. Confirm target after load profile is known.	Load test with documented data volume and hardware.
Freshness	Expose per-source freshness SLA; no universal cadence assumed across providers.	Stale-feed integration test.
Scalability	Paginated APIs, indexed time/spatial queries, asynchronous ingestion for slow connectors.	Profile event and incident query paths.
Reliability	Idempotent writes; bounded retries; dead-letter/quarantine path; deterministic replay.	Duplicate, failure, and replay tests.
Security	Auth for all writes; least privilege; secrets in environment/secret manager; TLS in deployment.	Permission and secret-leak checks.
Privacy	Collect minimum personal data; restrict citizen-report text/attachments; redact exports/logs as configured.	Privacy review and test fixtures.
Audit	Immutable-by-app audit trail for material actions/configuration changes.	Attempted modification + traceability tests.
Observability	Structured logs, metrics for ingest success/failure/lag, correlation IDs, health/readiness endpoints.	Dashboard/alert smoke tests.
Maintainability	Type hints, lint/format, migrations, automated tests, API docs, setup instructions.	CI checks and clean setup test.
Explainability	Scores and hypotheses carry evidence references, assumptions, version, and rationale.	Schema and UI acceptance review.

12.1 Safety and trust constraints

• No autonomous dispatch, public alert, or actuation in MVP.

• No fabricated live data. All demo/replay values must be labeled and traceable to a scenario or dataset.

• Do not infer that no incident exists merely because a feed is missing.

• Avoid absolute causal language unless a relationship has been validated through an approved process.

• Show uncertainty and contradictory evidence to the operator rather than hiding it to make the dashboard appear decisive.

• Recommendations must be reviewable, reversible where possible, and tied to an incident, rationale, and approval state.

12.2 Operational configuration

• Keep provider credentials and endpoints configurable by environment; provide .env.example with names/placeholders only.

• Configure polling cadence, timeouts, retry/backoff, freshness SLA, thresholds, spatial/time windows, and retention separately.

• Provide safe defaults for local demo, plus an explicit mode that prevents accidental calls to paid/restricted APIs.

• Log request IDs and connector run IDs; never log authorization headers, API keys, or raw personal information.

13. QA strategy and release acceptance

13.1 Minimum scenario matrix

Scenario	Expected result
Normal weather + traffic events	Events normalize, persist, and appear in Event Explorer; no anomaly unless rules warrant it.
Heavy rainfall + citizen report + rising water + traffic slowdown	Candidate incident formed with traceable supporting evidence and a cascade path; no claim of causal proof.
Duplicate provider event replayed twice	One canonical event; duplicate receipt trace; corroboration count unchanged.
Late/out-of-order water reading	Accepted with event/ingest times preserved; affected candidate is reconciled.
Contradictory nearby sensors	Conflict is visible; confidence/review state reflects contradiction; evidence is not silently dropped.
Offline/stale source	Source health changes; affected view shows stale/degraded; unrelated connectors continue.
Invalid coordinate / timestamp / unit	Rejected or quarantined with reason; no silent ingestion.
Two unrelated incidents close in time	Candidate matcher does not force merge; alternatives or separate incidents remain.
Agent timeout / malformed output	Core ingestion continues; agent task fails safely and produces fallback/visible status.
Unauthorized action approval	Server denies request and records security-relevant event without leaking data.
Synthetic replay mode	All relevant pages show SYNTHETIC/REPLAY provenance and scenario clock.
Merge/split/resolution	Action requires appropriate permission/reason and produces audit record.

13.2 Definition of Done

• Requirement implemented and linked to a test or explicit acceptance walkthrough.

• API contract and data schema documented; migrations applied cleanly from an empty database.

• Loading, empty, degraded, and error states implemented for the affected screen.

• Source provenance and uncertainty are visible wherever a signal informs an incident or recommendation.

• Automated tests pass; critical scenario matrix passes on seeded replay data.

• No credentials/secrets committed; permissions enforced server-side.

• README covers local setup, environment variables, database setup, seeded data, replay scenario, tests, and demo walkthrough.

• Product owner signs off on the acceptance criteria and unresolved assumptions.

13.3 Proposed MVP acceptance thresholds

The following are starting targets, not claims about current performance. Tune after profiling and a labelled replay set is available.

Measure	Proposed acceptance
Connector coverage	Five source categories supported via live connector or explicitly labeled replay/synthetic adapter.
Idempotency	100% of defined duplicate fixtures result in one canonical event.
Invalid data	All invalid fixtures receive explicit reject/quarantine reason; none silently accepted as valid.
Late events	All defined out-of-order fixtures retained and linked/reconciled within the documented replay workflow.
Explainability	Every generated incident candidate/hypothesis exposes evidence IDs, rationale, and confidence dimension.
Failure isolation	Failure of one connector does not block unrelated source ingestion or read APIs.
Security	Role tests prove restricted writes are rejected by API, not only hidden in UI.
Reproducibility	Fixed scenario/config/version yields stable event and incident outcomes, except explicitly nondeterministic model text.

14. Delivery plan, dependencies, and work breakdown

14.1 Suggested implementation sequence

Phase	Work packages	Exit criteria
0. Product/design foundation	Confirm personas, incident taxonomy, status model, confidence definitions, wireframes, data provenance labels.	PRD decisions signed; API/event contract reviewed.
1. Platform foundation	FastAPI, config/logging, database migrations, health, auth skeleton, CI, seed fixtures.	Clean setup works; health and schema tests pass.
2. Ingestion	Base adapter contract; weather/AQ/traffic/water/citizen adapters; replay mode; raw/normalized storage.	Five categories ingest or replay with provenance.
3. Deterministic processing	Validation, quarantine, idempotency, anomaly/source health, candidate retrieval, reconciliation.	Scenario matrix for duplicates, invalids, late events, outages passes.
4. Core operator UI	Global shell, overview, operations map/list, incident detail, event explorer, source health.	Operator journey works end-to-end on replay dataset.
5. Evidence & cascade	Evidence graph, hypotheses, contradiction display, asset dependencies, scenario comparison.	Each result traceable to evidence/assumptions; no causal overclaim.
6. Response + audit	Action queue, approval states, role enforcement, audit history.	Mutating actions permission-checked and audited.
7. Hardening/demo	Performance checks, accessibility pass, failure handling, docs, demo script, deployment.	Definition of Done and release gates passed.

14.2 Team ownership suggestion

Workstream	Accountable role	Typical deliverables
Product / domain	Product owner	Incident taxonomy, priority rules, acceptance decisions, response catalogue.
UX/UI	Product designer	Figma screens, component states, interactions, accessibility checks.
API / platform	Backend engineer	FastAPI, auth, event/incident/action APIs, persistence, audit.
Data ingestion	Data engineer	Connectors, normalization, replay data, source health, data lineage.
Fusion / graph	Data/graph engineer	Candidate scoring, evidence links, Neo4j schema, graph queries.
AI workflows	AI engineer	LangGraph orchestration, structured outputs, guardrails, evals.
QA / release	QA/DevOps	Test scenarios, CI, deployment, monitoring, reproducible demo.

14.3 MVP versus production pilot

Capability	MVP / demo	Production pilot requires additional
Data sources	Live where access allows; replay/synthetic otherwise, visibly labeled.	Provider agreements, SLAs, quota handling, source-specific validation and monitoring.
Fusion	Transparent heuristics, curated replay scenarios, analyst override.	Labelled ground truth, threshold tuning, precision/recall measurement, drift review.
Causal analysis	Competing hypotheses and explicit assumptions.	Domain review, causal validation methodology, calibrated uncertainty, model governance.
Response actions	Human-reviewed recommendations, no execution.	Operational SOP approval, escalation ownership, safety case, integration testing.
Security	Role checks, secret hygiene, audit.	Threat modelling, penetration testing, formal retention/privacy review, incident response plan.
Reliability	Single deployment with failure isolation and replay.	HA, backup/restore tests, monitoring/on-call, disaster recovery objectives.

15. Risks, open decisions, and governance

Risk / decision	Impact	Mitigation / required decision
Provider access / API quotas	Live signals may be unavailable or inconsistent during demo.	Implement replay-first architecture; label mode; document connector coverage and limitations.
Unlabelled historical data	Fusion score quality cannot be measured reliably.	Create curated scenario fixtures; label expected incident groupings and contradictions.
False incident merges	Operators may lose distinction between separate events.	Retain alternatives; require score breakdown; add human review and audited split.
False causal certainty	Users may treat correlation as proof.	Label relationship semantics; show assumptions, counter-evidence, and confidence dimension.
Missing sensors	System may mistake absence of data for absence of risk.	Expose freshness/expected-signal state; never equate missing with zero/safe.
Agent hallucination / invalid output	Unsupported cause or action recommendation.	Structured schema, evidence IDs, validators, bounded retries, fallback and human approval.
Graph/database divergence	UI may show stale or inconsistent relationships.	Postgres system of record; idempotent graph writes; outbox/reconciliation job.
Sensitive citizen data	Privacy or trust harm.	Minimize collection, redact logs/exports, restrict raw text/attachments, define retention.
Thresholds are uncalibrated	Severity and ranking may be misleading.	Mark as heuristic; version configuration; evaluate on labelled scenarios before production.
Operational action ownership unclear	Recommendations may not lead to coordinated response.	Define action owner, approval roles, escalation SLA, and approved response catalogue.

15.1 Decisions to resolve before implementation freeze

Decision	Default recommendation	Owner / status
Initial city and geography	Mumbai-first; geographic bounds configurable.	TBD
Incident taxonomy and severity	Approve a small, documented set before scoring/UI work.	TBD
Confidence definitions and scale	Separate validity, membership, cause; use uncalibrated heuristic labels until evaluated.	TBD
Authentication provider	Use the team's standard auth; local demo accounts only in explicit demo mode.	TBD
Data retention and citizen-report privacy	Minimize personal data; define durations before production use.	TBD
Map provider and tile/licence	Select an approved provider; comply with attribution/licensing.	TBD
Live connector commitments	Confirm keys, quotas, permitted use, and acceptable replay fallback.	TBD
Action catalogue and approval levels	Start with non-executing suggestions and supervisor approval.	TBD
Performance target / expected data volume	Profile demo fixture, then confirm numeric SLOs.	TBD
Production deployment target	Choose cloud/hosting, secrets management, backups, and monitoring.	TBD

15.2 Governance rules

- Product owner approves taxonomy, severity definitions, response catalogue, and scope changes.
- Data owner approves source use, retention, licensing, and sensitive fields.
- Engineering owner approves schema/API changes and migration plan.
- AI/graph owner documents model/prompt/config versions and evaluation results.
- QA/release owner may block release for failed critical scenarios, unsafe data presentation, or permission defects.

16. Glossary and implementation checklist

Term	Meaning in this product
Canonical event	Normalized, validated event record with stable identity and source provenance.
Incident candidate	Provisional grouping of events that may describe the same real-world incident.
Incident	Operator-visible record representing an event or grouped set of evidence, with status and ownership.
Event validity confidence	Estimate of whether a signal/event itself is credible.
Incident-membership confidence	Estimate that a given event belongs to a specific incident.
Cause-hypothesis confidence	Estimate supporting one explanation of the incident; distinct from membership confidence.
Supporting evidence	Evidence that is compatible with a claim/hypothesis under defined rules.
Contradictory evidence	Evidence that challenges a claim/hypothesis or indicates a competing explanation.
Cascade risk	Potential downstream effects along infrastructure/hazard dependencies over a stated horizon.
Source health	Freshness, connectivity, quality, and error status of a connector/feed.
Replay mode	Controlled playback of a fixed dataset/scenario; not represented as live data.
Deterministic processing	Code-driven validation, normalization, deduplication, thresholds, and candidate retrieval with reproducible behavior.
Evidence graph	Graph of events, incidents, assets, hypotheses, and typed relationships with provenance.
Outbox/reconciliation	Pattern for reliably propagating committed operational changes to another store such as Neo4j.

16.1 Build-ready checklist

■ Product owner confirms the nine-page navigation and screen priorities.

■ Incident taxonomy, status transitions, severity thresholds, and confidence semantics are written down.

■ Canonical event schema and API contract are approved and versioned.

■ Every source adapter documents mode, cadence, units, licence/access, failure behavior, and provenance.

■ Replay dataset contains normal, high-risk, contradictory, duplicate, late, invalid, and outage scenarios.

■ Postgres constraints and migrations enforce idempotency; graph writes can be retried/reconciled.

■ Overview, map/list, incident detail, evidence, cascade, response, event explorer, source health, and settings/audit screens implement all states.

■ AI outputs are structured, validated, evidence-linked, versioned, and fail safely.

■ All high-impact actions are human-reviewed; no automatic real-world dispatch in MVP.

■ QA runs the scenario matrix; product owner signs off; setup/demo/recovery instructions are documented.

16.2 Suggested demo walkthrough

1. Start with a labeled synthetic/replay scenario showing rainfall, a citizen report, water-level change, and traffic slowdown.

2. Show Overview: source freshness, active incident candidate, and top cascade risk.

3. Open Live Operations and select the candidate; point out time and location filters.

4. Open Incident Detail and explain the separate validity, membership, and cause confidence dimensions.

5. Open Evidence & Causal Analysis; show supporting evidence, contradictory evidence, and at least two competing hypotheses.

6. Open Cascade Risk; compare baseline with one or more interventions, including assumptions and unknowns.

7. Create a response action and route it for human approval; show audit history.

8. Open Event Explorer and Data Sources to prove event lineage, duplicate handling, and replay provenance.

9. Simulate a source outage or late event and show that source health and incident uncertainty update without the system claiming safety.

End of PRD • Version 1.0 • This document is the baseline specification; TBD decisions must be resolved and recorded before a production pilot.